UNIVERSITÀ DI PARMA

Dipartimento di Scienze Matematiche, Fisiche e Informatiche

Corso di Laurea Triennale in Informatica

Analisi malware con Cuckoo Sandbox

Candidato:
Ludovico Gorrieri

Matricola:
332524

Relatore:
Roberto Alfieri

Anno Accademico 2021–2024

Indice

1 Introduzione **3**

2 Wazuh **5**
2.0.1 Cos'è Wazuh 5
2.0.2 Wazuh Agent 6
2.0.3 Wazuh Server 8

3 Strumenti per l'analisi dinamica **11**

4 Realizzazione di un server per l'analisi dinamica **15**

5 Analisi di un data-set di malware **31**
5.1 Invio dei campioni e struttura delle analisi 31
5.2 Dettagli delle analisi svolte e informazioni estratte 33

6 Risultati **39**

Bibliografia **45**

Elenco delle figure

2.1 Struttura di Wazuh[5] 6

5.1 Esempio dell'interfaccia web di Cuckoo per l'invio dei campioni. 31
5.2 Distribuzione richieste HTTP 34
5.3 Protocolli per utilizzo 34
5.4 Distribuzione dei domini più contattati 35
5.5 Distribuzione delle famiglie di comportamenti rilevate 35
5.6 Distribuzione chiamate ad API crypto 36
5.7 Distribuzione dei processi più frequentemente rilevati 36
5.8 Distribuzione dei file con più letture e scritture 37

6.1 Statistiche sui tempi di analisi 39
6.2 Distribuzione dei punteggi di rischio 40
6.3 Falsi negativi e intervalli dei punteggi 40
6.4 Distribuzione numero di signatures 41
6.5 Eventi generati da Cuckoo su Wazuh 41
6.6 Esempio di grafico generato su Wazuh riguardante gli eventi generati da Cuckoo 42
6.7 Esempio di evento Cuckoo correttamente generato su Wazuh . . 42

Tabella degli Algoritmi

4.1 installazione dei prerequisiti 15
4.2 configurazione dell'utente Cuckoo 16
4.3 cuckoo-setup-virtualenv.sh 16
4.4 installazione di Cuckoo 17
4.5 creazione e configurazione della macchina virtuale 17
4.6 cuckoo.conf 18
4.7 memory.conf 19
4.8 reporting.conf 22
4.9 virtualbox.conf 22
4.10 installazione dell'agent Wazuh su Cuckoo server 23
4.11 jsondump.py 24
4.12 /var/ossec/etc/ossec.conf 26
4.13 sezione di ossec.conf sull'agent dell'endpoint 27
4.14 malicious_response.exe 27
4.15 /var/ossec/etc/ossec.conf 28
4.16 cuckoo-decoder.xml 28
4.17 avvio di Cuckoo 29
5.1 esempio di utilizzo dell'API Cuckoo 31
5.2 esempio di utilizzo di Cuckoo da CLI 31

Capitolo 1

Introduzione

La rapida e incontrollata evoluzione delle tecnologie negli ultimi anni ha portato ad una espansione del settore informatico, ad uno spostamento dei dati sensibili di aziende e enti pubblici in formati digitali e alla implementazione e utilizzo di ecosistemi sempre più interconnessi. Questi avanzamenti hanno portato a un miglioramento delle prestazioni e della produttività dei settori coinvolti ma, come diretta conseguenza, a nuove necessità e problematiche legate alla gestione e sicurezza delle risorse sensibili di utenti ed entità. Con lo sviluppo si è potuto anche assistere a una impennata delle attività criminali nel settore informatico portando alla impellente necessità di gestire eventi legati alla sicurezza in maniera rapida, automatizzata e, sopratutto, documentabile. La generazione di informazioni composte e complesse estratte da varie forme di monitoraggio e intelligence è di estrema importanza per una prevenzione predittiva degli asset.

I **SIEM** - (*System Information and Event Management*) sono soluzioni software che si occupano di raccogliere dati da vari endpoint (logs, traffico di rete, filesystem...) e di aggregarli al fine di individuare eventi di sicurezza, mettendo a disposizione azioni predefinite e personalizzabili per la creazione di intelligence e per rispondere adeguatamente alle minacce (invio di email automatico, scripts di active response...). Questi potenti strumenti sono di estrema importanza per sanare le problematiche odierne del settore e presentano numerose soluzioni, open source o commerciali.

Esattamente come per tutte le tipologie di attacchi informatici i malware stanno subendo una rapidissima evoluzione che rende il riconoscimento tramite analisi statica di eventuali file pericolosi sempre meno affidabile.
Sta diventando sempre più difficile individuare campioni malevoli a causa di nuove tecniche di offuscamento, continuo mutamento, ibridazione dei ceppi e un mercato del malware sempre più aggressivo e redditizio.

L'**analisi dinamica** del software potrebbe essere una buona soluzione per individuare attività potenzialmente pericolose dato un campione. Queste tecniche permettono di analizzare il comportamento di file in maniera automatica tramite l'uso di sandbox isolate dove viene messo in esecuzione e monitorato al fine di generare reports dettagliati su tutte le attività messe in atto dal campione. Analizzando il comportamento si ha la possibilità di stare al passo con le tecniche di offuscamento e la comparsa di nuovi malware. Questi strumenti sono altamente scalabili e, nonostante impieghino più tempo per analisi, raggiungono un livello di dettaglio e una percentuale di corretto riconoscimento di malware notevole.
L'integrazione, inoltre, delle informazioni date dalle analisi con un SIEM permetterebbe di gestire ogni analisi come evento, aggregando tanti dati utili all'intelligence e alla creazione di informazioni predittive.
L'obiettivo di questo studio è di esplorare gli strumenti di analisi dinamica automatica in modo da trovare un candidato adeguato all'integrazione con Wazuh (SIEM universitario), testare un dataset di campioni malware in modo da generare statistiche utili alla stima del beneficio dato dall'utilizzo e predisporre un endpoint per la raccolta e invio automatici di campioni utilizzando il modulo di Integrity Monitoring di Wazuh.

I passaggi delineati per raggiungere questo scopo sono seguenti:

- studio dell'ecosistema Wazuh.
- scelta della strumentazione per l'analisi dinamica automatica.
- realizzazione di un server adibito alle analisi.
- integrazione del server con Wazuh e creazione di uno script per la raccolta automatica di campioni.
- analisi di un dataset di malware e aggregazione dei dati derivanti dalle analisi al fine di ottenere delle metriche sufficienti a giudicare l'efficienza del progetto.

Capitolo 2

Wazuh

Come accennato in precedenza Wazuh è un **SIEM** che offre una moltitudine di funzionalità.
Tra queste troviamo i moduli di **File Integrity Monitoring** e di **Active Response** che combinati aprono la possibilità di ottenere un endpoint in grado di fornirci campioni di file derivanti dalle attività di utilizzo della macchina da parte di utenti anche molto eterogenei, molto utili per performare analisi su di essi in grado di fornirci informazioni anche di stampo predittivo sulle difese che le macchine nella rete di interesse necessitano. Prima di vedere nel dettaglio le funzionalità di questi due moduli, verrà data una breve introduzione a Wazuh.

2.0.1 Cos'è Wazuh

Wazuh e' una piattaforma open source che offre le funzionalita' di **SIEM** e di **XDR** (Extended Detection and Response) utilizzabile per la rilevazione delle anomalie e protezione dalle potenziali minacce sugli endpoints [4] (containers Docker compresi). Questo strumento viene utilizzato per monitorare vari endpoint sotto molteplici punti vista tra cui traffico di rete e integrità di sistema, delineare eventi di sicurezza tracciandoli nel tempo e aggregare dati in maniera complessa e strutturata tramite le dashboard personalizzabili e accessibili dal web.
Un'altra caratteristica molto importante per l'implementazione di questo progetto è la possibilità di sviluppare scripts di active response (scripts personalizzati o già forniti dal produttore utilizzati per rispondere automaticamente ad eventi di sicurezza).
Wazuh è composto di quattro componenti principali:

- Agent
- Server

- Indexer
- Dashboard

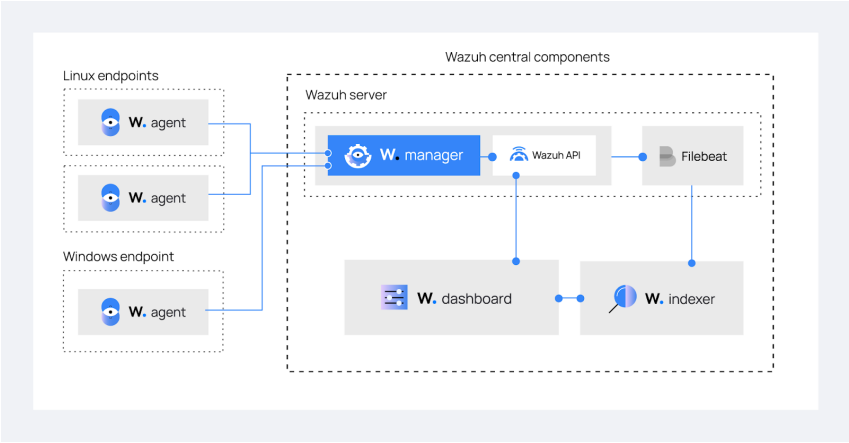

Figura 2.1: Struttura di Wazuh[5]

2.0.2 Wazuh Agent

L'agente contribuisce a proteggere il sistema fornendo capacità di prevenzione, rilevamento e risposta alle minacce. Inoltre, è utilizzato per raccogliere diversi tipi di dati di sistema e applicazioni, che vengono successivamente inoltrati al server Wazuh tramite un canale criptato e autenticato.[4] L'agente Wazuh presenta un'architettura modulare. Ogni componente è responsabile delle proprie funzioni, tra cui il monitoraggio del file system, la lettura dei messaggi di log, la raccolta di dati di inventario, la scansione della configurazione del sistema e la ricerca di malware[4]. Gli utenti possono gestire i moduli dell'agente tramite impostazioni di configurazione, adattando la soluzione ai propri casi d'uso specifici[4]. Tutti i moduli dell'agente sono configurabili e svolgono compiti di sicurezza differenti. Questa architettura modulare consente di abilitare o disabilitare ciascun componente in base alle proprie esigenze di sicurezza. Di seguito sono riportati i diversi scopi di tutti i moduli dell'agente.

- **Collettore di log:** Questo componente dell'agente è in grado di leggere file di log in formato flat e eventi di Windows, raccogliendo messaggi di log del sistema operativo e delle applicazioni. Supporta filtri XPath per gli eventi di Windows e riconosce formati multilinea come i log di audit di Linux. Inoltre, può arricchire eventi JSON con metadati aggiuntivi.
- **Esecuzione di comandi:** Gli agenti eseguono periodicamente comandi autorizzati, raccogliendo il loro output e riportandolo al server Wazuh

per ulteriori analisi. Questo modulo può essere utilizzato per vari scopi, come monitorare lo spazio su disco disponibile o ottenere un elenco degli utenti che si sono recentemente collegati.

- **Monitoraggio dell'integrità dei file (FIM):** Questo modulo monitora il file system, segnalando quando i file vengono creati, eliminati o modificati. Tieni traccia delle modifiche negli attributi dei file, nelle autorizzazioni, nella proprietà e nel contenuto. Quando si verifica un evento, cattura in tempo reale chi, cosa e quando. Inoltre, il modulo FIM costruisce e mantiene un database con lo stato dei file monitorati, consentendo di eseguire query da remoto.

- **Valutazione della configurazione di sicurezza (SCA):** Questo componente fornisce una valutazione continua della configurazione, utilizzando controlli predefiniti basati sui benchmark del Center for Internet Security (CIS). Gli utenti possono anche creare i propri controlli SCA per monitorare e applicare le proprie politiche di sicurezza.

- **Inventario del sistema:** Questo modulo dell'agente esegue scansioni periodiche, raccogliendo dati di inventario come la versione del sistema operativo, le interfacce di rete, i processi in esecuzione, le applicazioni installate e un elenco delle porte aperte. I risultati della scansione vengono memorizzati in database SQLite locali che possono essere interrogati da remoto.

- **Rilevamento di malware:** Utilizzando un approccio non basato su firme, questo componente è in grado di rilevare anomalie e la possibile presenza di rootkit. Inoltre, cerca processi nascosti, file nascosti e porte nascoste monitorando le chiamate di sistema.

- **Active response:** Questo modulo esegue azioni automatiche quando vengono rilevate minacce, attivando risposte per bloccare una connessione di rete, fermare un processo in esecuzione o eliminare un file dannoso. Gli utenti possono anche creare risposte personalizzate quando necessario e personalizzare, ad esempio, le risposte per eseguire un file binario in una sandbox, catturare il traffico di rete e scansionare un file con un antivirus.

- **Monitoraggio della sicurezza dei containers:** Questo modulo dell'agente è integrato con l'API del Docker Engine per monitorare le modifiche in un ambiente containerizzato. Ad esempio, rileva modifiche alle immagini dei contenitori, alla configurazione di rete o ai volumi di dati. Inoltre, genera avvisi su contenitori in esecuzione in modalità privilegiata e su utenti che eseguono comandi in un contenitore in esecuzione.

- **Monitoraggio della sicurezza nel cloud:** Questo componente monitora fornitori di cloud come Amazon Web Services, Microsoft Azure o Google GCP. Comunica nativamente con le loro API. È in grado di rilevare modifiche all'infrastruttura cloud (ad esempio, la creazione di un nuovo utente, la modifica di un gruppo di sicurezza, l'arresto di un'istanza cloud, ecc.) e raccogliere dati di log dei servizi cloud (ad esempio, AWS Cloudtrail, AWS Macie, AWS GuardDuty, Azure Active Directory, ecc.).

L'agente Wazuh comunica con il server Wazuh per inviare i dati raccolti e gli eventi relativi alla sicurezza. Inoltre, l'agente invia dati operativi, riportando la propria configurazione e stato. Una volta connesso, l'agente può essere aggiornato, monitorato e configurato da remoto dal server Wazuh. La comunicazione dell'agente con il server avviene tramite un canale sicuro (TCP o UDP), fornendo crittografia e compressione dei dati in tempo reale. Inoltre, include meccanismi di controllo del flusso per evitare il flooding, mettendo in coda gli eventi quando necessario e proteggendo la larghezza di banda della rete. È necessario registrare l'agente prima di collegarlo al server per la prima volta. Questo processo fornisce all'agente una chiave unica utilizzata per l'autenticazione e la crittografia dei dati.

2.0.3 Wazuh Server

Il componente server di Wazuh analizza i dati ricevuti dagli agenti, attivando avvisi quando vengono rilevati minacce o anomalie. Viene utilizzato anche per gestire la configurazione degli agenti in remoto e monitorarne lo stato. Il server Wazuh utilizza fonti di intelligence sulle minacce per migliorare le proprie capacità di rilevamento. Arricchisce inoltre i dati degli avvisi utilizzando il framework **MITRE ATT&CK** e requisiti di conformità normativa come **PCI DSS, GDPR, HIPAA, CIS e NIST 800-53**, fornendo un contesto utile per l'analisi della sicurezza[4]. In aggiunta, il server può essere integrato con software esterni, inclusi sistemi di gestione dei ticket come ServiceNow, Jira e PagerDuty, oltre a piattaforme di messaggistica istantanea come Slack. Queste integrazioni sono convenienti per semplificare le operazioni di sicurezza.
Esso esegue il motore di analisi, l'API RESTful di Wazuh, il servizio di registrazione degli agenti, il servizio di connessione degli agenti, il daemon del cluster Wazuh e Filebeat[4]. Il server è installato su un sistema operativo Linux e generalmente gira su una macchina fisica autonoma, una macchina virtuale, un contenitore Docker o un'istanza cloud. Il server Wazuh è composto da diversi componenti elencati di seguito, ognuno con funzioni differenti, come registrare nuovi agenti, convalidare l'identità di ciascun agente e criptare le comunicazioni tra l'agente Wazuh e il server.

- **Servizio di registrazione degli agenti:** Viene utilizzato per registrare nuovi agenti. Questo servizio fornisce e distribuisce chiavi di autenticazione uniche a ciascun agente. Il processo opera come un servizio di rete e supporta l'autenticazione tramite certificati TLS/SSL o fornendo una password fissa.

- **Servizio di connessione degli agenti:** Questo servizio riceve i dati dagli agenti. Utilizza le chiavi condivise dal servizio di registrazione per convalidare l'identità di ciascun agente e criptare le comunicazioni tra l'agente Wazuh e il server. Inoltre, questo servizio fornisce una gestione centralizzata della configurazione, permettendo di inviare nuove impostazioni per gli agenti in remoto.

- **Motore di analisi:** Questo è il componente del server che esegue l'analisi dei dati. Utilizza decodificatori per identificare il tipo di informazione che viene elaborata (eventi Windows, log SSH, log di server web e altri). Questi decodificatori estraggono anche elementi di dati rilevanti dai messaggi di log, come indirizzo IP sorgente, ID evento o nome utente. Successivamente, utilizzando regole, il motore identifica schemi specifici negli eventi decodificati che potrebbero attivare avvisi e, possibilmente, richiedere contromisure automatiche (ad esempio, bloccare un indirizzo IP, fermare un processo in esecuzione o rimuovere un artefatto di malware).

- **API RESTful di Wazuh:** Questo servizio fornisce un'interfaccia per interagire con l'infrastruttura di Wazuh. Viene utilizzato per gestire le impostazioni di configurazione di agenti e server, monitorare lo stato dell'infrastruttura e la salute generale, gestire e modificare decodificatori e regole di Wazuh, e interrogare lo stato degli endpoint monitorati. Anche il dashboard di Wazuh lo utilizza.

- **Demone del cluster Wazuh:** Questo servizio viene utilizzato per scalare i server Wazuh orizzontalmente, distribuendoli come un cluster. Questa configurazione, combinata con un bilanciatore di carico di rete, offre alta disponibilità e bilanciamento del carico. Il demone del cluster Wazuh è ciò che i server Wazuh utilizzano per comunicare tra loro e mantenere la sincronizzazione.

- **Filebeat:** Viene utilizzato per inviare eventi e avvisi all'indicizzatore Wazuh. Legge l'output del motore di analisi di Wazuh e invia gli eventi in tempo reale. Fornisce anche bilanciamento del carico quando connesso a un cluster di indicizzatori Wazuh multi-nodo.

Capitolo 3

Strumenti per l'analisi dinamica

Gli strumenti di analisi dinamica di malware sono progettati per esaminare il comportamento di un software sospetto durante la sua esecuzione, al fine di determinare attività dannose o non autorizzate che potrebbero compromettere un sistema informatico. Questi strumenti forniscono un ambiente controllato in cui il malware può essere eseguito in sicurezza, monitorando in tempo reale le interazioni del codice con il sistema operativo e altre risorse, senza rischiare di danneggiare il sistema reale.
In genere, un sistema di analisi dinamica crea un sandbox, cioè un ambiente isolato che simula un sistema operativo o una macchina virtuale, dove il malware può essere eseguito senza poter interferire con il sistema host. Durante questa fase, vengono raccolti dati riguardanti una serie di attività, come la modifica di file di sistema, la comunicazione di rete, l'utilizzo di API, la creazione di nuovi processi e l'esecuzione di operazioni di registrazione su disco o registro di sistema.
L'obiettivo principale di questi strumenti è identificare comportamenti anomali che suggeriscono azioni dannose, come il furto di dati, la propagazione del malware, l'esecuzione di comandi remoti o l'interferenza con la sicurezza del sistema. Queste informazioni sono cruciali per comprendere la natura del malware, la sua modalità di infezione e le potenziali vulnerabilità del sistema target. L'analisi dinamica, sebbene potente, viene spesso utilizzata in combinazione con tecniche di analisi statica per ottenere una visione completa e più accurata del comportamento del codice maligno.
In sintesi, gli strumenti di analisi dinamica di malware offrono un metodo efficace per osservare e registrare i comportamenti di software sospetti in un ambiente sicuro, supportando così le indagini sulla sicurezza informatica e la difesa contro minacce avanzate.
I sistemi di analisi dinamica più utilizzati sono:

Cuckoo Sandbox:

Cuckoo Sandbox è uno strumento open-source di analisi dinamica di malware, progettato per esaminare il comportamento di file sospetti e applicazioni malevole. Le sue capacità si estendono a un'ampia gamma di tipologie di file, inclusi eseguibili, documenti, script, URL e file di posta elettronica.
Il sistema consente l'esecuzione di campioni di malware in un ambiente controllato, come macchine virtuali o sistemi reali isolati, monitorando le attività del malware in tempo reale e registrando ogni azione intrapresa.

Le principali funzionalità di Cuckoo Sandbox includono:

- **Monitoraggio delle Interazioni del Malware con il Sistema**: Cuckoo raccoglie dati sull'utilizzo delle API, la creazione di nuovi processi, le modifiche ai file di sistema e le scritture nel registro di sistema[2].
- **Analisi del Traffico di Rete**: La piattaforma esamina e registra tutto il traffico di rete generato dal malware, incluse le comunicazioni HTTP, DNS e altre connessioni di rete, per individuare tentativi di contatto con server remoti o infrastrutture di comando e controllo[2].
- **Reportistica Dettagliata**: Al termine di ogni analisi, Cuckoo genera rapporti completi che includono sia dati di rete sia informazioni comportamentali. Questi rapporti possono essere prodotti in vari formati, tra cui JSON e HTML, per facilitare ulteriori analisi e integrazioni[2].
- **Integrazione con Altri Strumenti**: Cuckoo Sandbox può essere integrato in flussi di lavoro esistenti e si integra con strumenti di analisi di terze parti come YARA e Volatility per analisi forensi e di rilevamento delle firme[2].

Grazie alla sua flessibilità e alla possibilità di personalizzazione, Cuckoo è ampiamente utilizzato per individuare e analizzare una vasta gamma di minacce malware, rappresentando uno strumento fondamentale in numerosi contesti di sicurezza informatica.

CAPEv2:

CAPE (Config And Payload Extraction) v2 è un'evoluzione di Cuckoo Sandbox progettata specificamente per migliorare l'analisi di campioni di malware che utilizzano tecniche di evasione e per estrarre configurazioni e payload in modo dettagliato. Le sue capacità avanzate permettono non solo di osservare

il comportamento del malware, ma anche di identificarne e analizzarne più a fondo i meccanismi di funzionamento e di evasione.

Le funzionalità distintive di CAPEv2 includono:

- **Estrazione Automatica di Payload e Configurazioni**: CAPEv2 è specializzato nell'estrazione automatica di payload e configurazioni da vari tipi di malware, tra cui trojan bancari, ransomware e altre forme di malware che utilizzano la cifratura o tecniche di occultamento[1].
- **Esecuzione in Modalità Stealth**: Per contrastare i tentativi di evasione da sandbox, CAPEv2 include misure di stealth che simulano meglio un ambiente reale, rendendosi più difficile da rilevare per il malware analizzato[1].
- **Rilevamento e Mitigazione dell'Evasione**: CAPEv2 incorpora tecniche avanzate per rilevare e neutralizzare tentativi di evasione, come l'iniezione di codice in processi legittimi, la rilevazione di virtualizzazioni, e il mascheramento di comportamenti malevoli[1].
- **Integrazione Avanzata con Strumenti di Analisi**: CAPEv2 supporta plugin di analisi che espandono le capacità di base, permettendo agli analisti di includere moduli per la decrittazione di configurazioni e per l'analisi automatizzata di minacce persistenti avanzate (APT)[1].

Con le sue funzionalità focalizzate su payload e configurazioni, CAPEv2 è particolarmente utile per i ricercatori di sicurezza e gli analisti di malware che si occupano di minacce avanzate e sofisticate, fornendo strumenti dedicati per comprendere più a fondo la struttura e gli obiettivi di campioni dannosi.

PANDA:

PANDA (Platform for Architecture-Neutral Dynamic Analysis) è una piattaforma avanzata per l'analisi dinamica di malware e la retroazione sistematica, sviluppata per facilitare esperimenti di sicurezza informatica e monitorare il comportamento dei campioni in ambienti virtualizzati. Diversamente da altri strumenti di analisi, PANDA è stato concepito come un framework per l'analisi e la registrazione architecture-neutral, permettendo la riproduzione e l'analisi dettagliata di scenari in ambienti complessi.

Le capacità principali di PANDA includono:

- **Registrazione e Riproduzione dei Comportamenti**: PANDA permette di registrare interamente l'esecuzione di un sistema, salvando ogni

istruzione eseguita. Questo consente di riprodurre in modo preciso l'analisi e di eseguire studi approfonditi anche in un secondo momento[3].

- **Compatibilità Multi-architettura**: Progettato per essere compatibile con diverse architetture, PANDA permette di analizzare malware su più piattaforme, incluse x86, x86-64, ARM e altre, rendendolo ideale per ambienti di ricerca e testing su sistemi diversi[3].

- **Analisi Avanzata tramite Plugin**: PANDA supporta numerosi plugin per espandere le sue funzionalità, come la ricostruzione del flusso di dati, l'analisi del comportamento di codice specifico e l'osservazione delle interazioni con il sistema operativo. Alcuni plugin avanzati permettono persino l'iniezione di codice per osservare le reazioni del malware a specifiche condizioni[3].

- **Analisi Post-mortem e Diagnosi**: Grazie alla sua capacità di riproduzione, PANDA permette un'analisi post-mortem dettagliata, in cui gli analisti possono esaminare attentamente il comportamento del malware in vari punti dell'esecuzione, favorendo una comprensione granulare degli effetti delle istruzioni malevole sul sistema[3].

PANDA è particolarmente adatto in contesti di ricerca e sviluppo avanzati, in cui è necessario avere un controllo fine sui processi di analisi. La sua capacità di registrazione e riproduzione, unita alla flessibilità architetturale, lo rendono uno strumento versatile per chi necessita di una piattaforma di analisi dettagliata, completa e personalizzabile.

Al fine della realizzazione di questo progetto la scelta è ricaduta su Cuckoo Sandbox per la sua progettazione modularizzata e semplicità (molte delle caratteristiche degli altri strumenti sono troppo avanzate rendendole pressochè superflue o richiedono una diretta interazione dell'utente non adatta a un sistema che punta ad essere completamente automatizzato).

Capitolo 4

Realizzazione di un server per l'analisi dinamica

In questo capitolo verrà descritta l'implementazione di un server adibito alla ricezione, processamento di richieste di analisi e creazione di report automatici. I passaggi sono i seguenti:

- Installazione di Cuckoo server, dei suoi requisiti e dei moduli.
- preparazione di una sandbox.
- creazione di servizi dedicati a server, API e interfaccia web (systemctl).

La macchina utilizzata per questo progetto è una macchina virtuale con sistema operativo Ubuntu 22.04.4 LTS x86_64, eseguita su piattaforma VMware, dotata di 4 GB di RAM, processore Intel Xeon Gold 5120 a 2.20 GHz e disco virtuale da 107 GB. Cuckoo supporta in maniera stabile Python 2.7 Le librerie richieste per l'installazione di Cuckoo sono:

```
sudo apt-get install python python-pip  python-dev
   libffi-dev libssl-dev -y
sudo apt-get install python-virtualenv python-setuptools
   -y
sudo apt-get install libjpeg-dev zlib1g-dev swig -y
sudo apt-get install mongodb -y
sudo apt-get install postgresql libpq-dev -y
sudo apt install virtualbox -y
sudo apt-get install tcpdump apparmor-utils -y
sudo apt-get install swig
sudo pip install m2crypto
```

Codice 4.1: installazione dei prerequisiti

Installati i requisiti è necessario configurare un utente apposito:

```
sudo adduser --disabled-password --gecos "" cuckoo

#autorizza l'utente ad usare tcpdump
sudo groupadd pcap
sudo usermod -a -G pcap cuckoo
sudo chgrp pcap /usr/sbin/tcpdump
sudo setcap cap_net_raw,cap_net_admin=eip /usr/sbin/
    tcpdump
getcap /usr/sbin/tcpdump
sudo aa-disable /usr/sbin/tcpdump

#autorizza l'utente alla gestione di macchine virtuali
sudo usermod -a -G vboxusers cuckoo

sudo su cuckoo
```

Codice 4.2: configurazione dell'utente Cuckoo

Script per la configurazione dell'ambiente:

```
#!/usr/bin/env bash

sudo apt-get -y install virtualenv
sudo apt-get -y install virtualenvwrapper

echo "source /usr/share/virtualenvwrapper/
    virtualenvwrapper.sh" >> ~/.bashrc

sudo apt-get -y install python3-pip

# auto  completamento per pip3
pip3 completion --bash >> ~/.bashrc

pip3 install --user virtualenvwrapper

echo "export VIRTUALENVWRAPPER_PYTHON=/usr/bin/python3"
    >> ~/.bashrc

echo "source ~/.local/bin/virtualenvwrapper.sh" >> ~/.
    bashrc

export WORKON_HOME=~/.virtualenvs

echo "export WORKON_HOME=~/.virtualenvs" >> ~/.bashrc

```

```
echo "export PIP_VIRTUALENV_BASE=~/.virtualenvs" >> ~/.bashrc
```

Codice 4.3: cuckoo-setup-virtualenv.sh

Utilizzo dello script e installazione di Cuckoo

```
#per sudoer si intende l'utente abilitato a sudo usato in precedenza
sudo -u *sudoer* cuckoo-setup-virtualenv.sh

source ~/.bashrc

#creazione di un ambientevirtuale
mkvirtualenv -p python2.7 cuckoo-test

pip install -U pip  setuptools

#installazione di Cuckoo tramite pip, in caso di problemi in seguito a volte installare da github risolve il problema
pip install -U cuckoo
```

Codice 4.4: installazione di Cuckoo

Creazione e configurazione di una virtual machine usando vmcloak:

```
#download della macchina virtuale e delle librerie necessarie
sudo wget "https://cuckoo.sh/win7ultimate.iso"
sudo mkdir /mnt/win7
sudo chown cuckoo:cuckoo /mnt/win7/
sudo mount -o ro,loop win7ultimate.iso /mnt/win7

sudo apt-get -y install build-essential libssl-dev libffi-dev python-dev genisoimage
sudo apt-get -y install zlib1g-dev libjpeg-dev
sudo apt-get -y install python-pip   python-virtualenv python-setuptools swig

#installazione di vmcloak  (manager di macchine virtuali)
pip install -U vmcloak

#creazione di una scheda di rete
vmcloak-vboxnet0
# Inizializza una macchina virtuale con il nome win7x64 usando l'immagine base win7x64base.
```

```
# I parametri specificano il numero di CPU, la memoria RAM, e la modalita‘ verbose.
vmcloak init --verbose --win7x64 win7x64base --cpus 2 --ramsize 2048

#clone di win7xbase
vmcloak clone win7x64base Windows7

vmcloak list deps

#internet explorer 11
vmcloak install Windows7 ie11

#creazione di uno snapshot chiamato 192.168.56.101
vmcloak snapshot --count 1 win7x64cuckoo 192.168.56.101

vmcloak list vms
```

Codice 4.5: creazione e configurazione della macchina virtuale

A questo punto i requirements, Cuckoo e la vm sono pronti e funzionanti. Ora verranno mostrati i file di setup modificati (solo le impostazioni modificate) rispetto alla configurazione predefinita (i file sono situati i .cuckoo/conf): **cuckoo.conf** gestisce i dettagli generali di Cuckoo

```
# Il gestore di macchine virtuali desiderato per il sandboxing.
[cuckoo]
machinery=virtualbox
[resultserver]
# Indirizzo del server che immagazzinera‘ i risultati in tempo reale prodotti dall‘analyzer
# e che aggreghera‘ i dati per la compilazione dell‘analisi finale.
ip = localhost
port = 2042
[remotecontrol]
# Abilita il controllo remoto delle macchine virtuali dall‘interfaccia web da parte dell‘utente.
enabled = yes
guacd_host = localhost
guacd_port = 4822
```

Codice 4.6: cuckoo.conf

memory.conf regola l'analisi dei dump di memoria della sandbox. Alcuni moduli di Volatility sono volutamente disabilitati per necessità legate alla performance.

```
# Profilo della macchina virtuale.
guest_profile = Win7SP1x64

# Cancella i memory dump generati una volta finite le analisi.
delete_memdump = yes

# Impostazione dei moduli Volatility (strumento per effettuare analisi dei dump di memoria).

# Cerca codice e dll nascosti o iniettati.
# http://code.google.com/p/volatility/wiki/CommandReferenceMal23#malfind
[malfind]
enabled = yes
filter = yes

# Elenca le API agganciate in modalita' utente e nello spazio kernel.
# http://code.google.com/p/volatility/wiki/CommandReferenceMal23#apihooks
[apihooks]
enabled = no
filter = yes

# Elenca i processi ufficiali. non rileva processi nascosti.
# http://code.google.com/p/volatility/wiki/CommandReference23#pslist
[pslist]
enabled = yes
filter = no

# Elenca i processi nascosti. Usa vari metodi per identificarli.
# http://code.google.com/p/volatility/wiki/CommandReferenceMal23#psxview
[psxview]
enabled = yes
filter = no

```

```
33 # Mostra le callback.
34 # http://code.google.com/p/volatility/wiki/
      CommandReferenceMal23#callbacks
35 [callbacks]
36 enabled = yes
37 filter = no
38
39 # Mostra la Iterrupt Descriptor Table (IDT).
40 # http://code.google.com/p/volatility/wiki/
      CommandReferenceMal23#idt
41 [idt]
42 enabled = yes
43 filter = no
44
45 # Mostra i timers.
46 # http://code.google.com/p/volatility/wiki/
      CommandReferenceMal23#timers
47 [timers]
48 enabled = no
49 filter = no
50
51 # Mostra messagehooks.
52 # http://code.google.com/p/volatility/wiki/
      CommandReferenceGui23#messagehooks
53 [messagehooks]
54 enabled = no
55 filter = no
56
57 # Mostra i Security Identifiers (SIDs).
58 # http://code.google.com/p/volatility/wiki/
      CommandReference23#getsids
59 [getsids]
60 enabled = yes
61 filter = no
62
63 # Mostra i privilegi.
64 # http://code.google.com/p/volatility/wiki/
      CommandReference23#privs
65 [privs]
66 enabled = yes
67 filter = no
68
69 # Mostra i dll caricati dai processi. Non mostra i dll
      nascosti.
```

```
70 # http://code.google.com/p/volatility/wiki/
   CommandReference23#dlllist
71 [dlllist]
72 enabled = yes
73 filter = yes
74
75 # Mostra gli handles aperti dei processi.
76 # http://code.google.com/p/volatility/wiki/
   CommandReference23#handles
77 [handles]
78 enabled = yes
79 filter = yes
80
81 # Mostra i dll caricati dai processi compresi quelli
   nascosti(sganciati dal PEB).
82 # http://code.google.com/p/volatility/wiki/
   CommandReferenceMal23#ldrmodules
83 [ldrmodules]
84 enabled = yes
85 filter = yes
86
87 # Cerca Mutexes (in tutto il sistema).
88 # http://code.google.com/p/volatility/wiki/
   CommandReference23#mutantscan
89 [mutantscan]
90 enabled = yes
91 filter = yes
92
93 # Elenca dispositivi e drivers.
94 # http://code.google.com/p/volatility/wiki/
   CommandReferenceMal23#devicetree
95 [devicetree]
96 enabled = yes
97 filter = yes
98
99 # Cerca servizi.
100 # http://code.google.com/p/volatility/wiki/
   CommandReferenceMal23#svcscan
101 [svcscan]
102 enabled = yes
103 filter = yes
104
105 # Cerca drivers del kernel (compresi quelli nascosti o
   non caricati).
```

```
# http://code.google.com/p/volatility/wiki/CommandReference23#modscan
[modscan]
enabled = yes
filter = yes
# Scan con YARA.
[yarascan]
enabled = yes
filter = yes

# Scan della SSDT.
[ssdt]
enabled = yes
filter = yes

# Scan della GDT.
[gdt]
enabled = yes
filter = yes

# Solo Windows XP.
[sockscan]
enabled = yes
filter = no

# Solo Windows 7/Vista.
[netscan]
enabled = yes
filter = no
```

Codice 4.7: memory.conf

reporting.conf specifica il formato dei report e i dettagli legati al database.

```
[mongodb]
enabled = yes
host = 127.0.0.1
port = 27017
```

Codice 4.8: reporting.conf

virtualbox.conf gestisce le impostazioni legate alla macchine virtuali.

```
[virtualbox]
# modalita' di esecuzione delle macchine virtuali (GUI o headless).
mode = headless

```

```
# Interfaccia di rete di default.
interface = vboxnet0

# Lista separata da virgole delle macchine virtuali
    assegnate a Cuckoo.
machines = win7

# [win7] fa riferimento al nome specificati in machines.
[win7]

# Nome della macchina virtuale.
label = Windows7

# Sistema operativo in uso.
# [windows/darwin/linux].
platform = windows

# Indirizzo IP della macchina virtuale.
ip = 192.168.56.101

# Snapshot usato per il reset della macchina virtuale (
    opzionale).
snapshot = 192.168.56.101
```

Codice 4.9: virtualbox.conf

Configurato Cuckoo, il passo successivo consiste nel installare l'agent Wazuh sul server, modificare la classe usata da Cuckoo per generare i report e il server Wazuh per leggere i log custom.

Installazione agent:

```
# Installazione della chiave GPG

curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH |
    gpg --no-default-keyring --keyring gnupg-ring:/usr/
    share/keyrings/wazuh.gpg --import && chmod 644 /usr/
    share/keyrings/wazuh.gpg

# Aggiunta della repository ad apt

echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] https
    ://packages.wazuh.com/4.x/apt/ stable main" | tee -a /
    etc/apt/sources.list.d/wazuh.list

apt-get update
```

```

# Installazione dell'agent Wazuh e impostazione della variabile globale WAZUH_MANAGER (punta al server Wazuh)

WAZUH_MANAGER="ip-server-Wazuh" apt-get install wazuh-agent

systemctl daemon-reload

systemctl enable wazuh-agent

systemctl start wazuh-agent
```

Codice 4.10: installazione dell'agent Wazuh su Cuckoo server

Classe jsondump.py modificata per generare un dump delle analisi in formato syslog (dump del json) dentro /home/cuckoo/.cuckoo/log/reports.log

```
import calendar
import datetime
import json
import os

from cuckoo.common.abstracts import Report
from cuckoo.common.exceptions import CuckooReportError

def default(obj):
  if isinstance(obj, datetime.datetime):
    if obj.utcoffset() is not None:
      obj = obj - obj.utcoffset()
    return calendar.timegm(obj.timetuple()) + obj.microsecond / 1000000.0
  raise TypeError("%r is not JSON serializable" % obj)

class JsonDump(Report):
  """Save analysis results in JSON format."""

  def erase_calls(self, results):
    """Temporarily remove calls from the report by replacing them with
    empty lists."""
    if self.calls:
      self.calls = None
      return

```

```
26        self.calls = []
27        for process in results.get("behavior", {}).get("
    processes", []):
28          self.calls.append(process["calls"])
29          process["calls"] = []
30
31      def restore_calls(self, results):
32        """Restore calls that were temporarily removed in
    the report by
33        replacing the calls with the original values."""
34        if not self.calls:
35          return
36
37        for process in results.get("behavior", {}).get("
    processes", []):
38          process["calls"] = self.calls.pop(0)
39
40      def run(self, results):
41        """Writes report.
42        @param results: Cuckoo results dict.
43        @raise CuckooReportError: if fails to write report.
44        """
45        # Determine whether we want to include the
    behavioral data in the
46        # JSON report.
47        if "json.calls" in self.task["options"]:
48          self.calls = int(self.task["options"]["json.calls
    "])
49        else:
50          self.calls = self.options.get("calls", True)
51
52        self.erase_calls(results)
53        #tmp = results
54        try:
55          filepath = os.path.join(self.reports_path, "
    report.json")
56          with open(filepath, "wb", buffering=1024*1024) as
     report:
57            json.dump(
58              results, report, default=default, sort_keys=
    False,
59              indent=self.options.indent, encoding="latin-1
    "
60            )
```

```
61          with open("/home/cuckoo/.cuckoo/log/reports.log
    ", "a+") as log:
62            lines = log.readlines()
63            if(len(lines) > 100):
64              with open("/home/cuckoo/.cuckoo/log/reports
    .log.tmp", 'w') as outfile:
65                for line in lines[1:]:
66                  outfile.write(line)
67                os.rename("/home/cuckoo/.cuckoo/log/
    reports.log.tmp", "/home/cuckoo/.cuckoo/log/reports.
    log")
68            now = datetime.datetime.now()
69            results1 ={"signatures": results["signatures"
    ]}
70            formatted_now = now.strftime("%Y %b %d %H:%M
    :%S")
71            log.write(formatted_now+" cuckoo-reporting: "
    )
72            json.dump(
73              results1, log, default=default, sort_keys=
    False ,
74              indent=None, encoding="latin-1"
75            )
76            log.write("\n")
77        except (TypeError, IOError) as e:
78          raise CuckooReportError("Failed to generate JSON
    report: %s" % e)
79        finally:
80          self.restore_calls(results)
```

Codice 4.11: jsondump.py

Configurazione del server Wazuh per la lettura degli eventi ricevuto dal server cukoo.

```
1 <!-- configurazione per l'utilizzo dello script python
    sull'endpoint.
2 quando la regola 321321 viene attivata il programma viene
     lanciato. -->
3 <command>
4   <name>malicious_response</name>
5   <executable>malicious_response.exe</executable>
6   <timeout_allowed>yes</timeout_allowed>
7 </command>
8
9
```

```
<active-response>
  <disabled>no</disabled>
  <command>malicious_response</command>
  <location>local</location>
  <rules_id>321321</rules_id>
  <timeout>20</timeout>
</active-response>
```

Codice 4.12: /var/ossec/etc/ossec.conf

A questo punto configuriamo un endpoint Windows per mandare in analisi qualunque nuovo file comparso nelle cartelle Downloads, Documents, Desktop, Music e Images e sottocartelle. Il modulo di FIM sull'agent è configurato per fare scattare la regola "file-added" ogni volta che un file viene aggiunto nelle posizioni specificate

```
<syscheck>
  <directories realtime="yes" whodata="yes">C:\Users\*\
   Desktop</directories>
  <directories realtime="yes" whodata="yes">C:\Users\*\
   Downloads</directories>
  <directories realtime="yes" whodata="yes">C:\Users\*\
   Music</directories>
  <directories realtime="yes" whodata="yes">C:\Users\*\
   Documents</directories>
  <directories realtime="yes" whodata="yes">C:\Users\*\
   Images</directories>
  <directories>%WINDIR%/System32</directories>
  <directories>%WINDIR%/SysWOW64</directories>
</syscheck>
```

Codice 4.13: sezione di ossec.conf sull'agent dell'endpoint

Lo script Python prende in input un file json contenente i dettagli della regola scattata, estrae il percorso del file, verifica lo Zone Identifier (identifica la provenienza del file) e lo manda in analisi usando l'API di Cuckoo.

```
import os
import sys
import json
import requests

def has_zone_identifier(file_path):
  try:
    zone_identifier_path = file_path + ':Zone.Identifier'
    b = os.path.exists(zone_identifier_path)
    os.remove(zone_identifier_path)
```

```
    return b
  except Exception as e:
    print(f"Errore: {e}")
    return False

def send_to_cuckoo(file_path):
  url = 'http://cuckoo.fis.unipr.it:8090/tasks/create/
   file'
  headers = {'Authorization': 'Bearer Mqo8ZrQIf3Ca-2Ln-
   f3RbQ'}
  files = {'file': open(file_path, 'rb')}
  response = requests.post(url, headers=headers, files=
   files)

def main():
  input_str = ""
  for line in sys.stdin:
    input_str = line
    break
  data = json.loads(input_str)
  path = data["parameters"]["alert"]["syscheck"]["path"]
  if(has_zone_identifier(path)):
    send_to_cuckoo(path)




if __name__ == "__main__":
  main()
```

Codice 4.14: malicious_response.exe

Una volta generato un report di Cuckoo nella posizione .cuckoo/logs/reports.log questa regola presente sull'agent installato sul server Cuckoo che monitora il file di log invia al server centrale l'alert.

```
<localfile>
  <log_format>syslog</log_format>
  <location>/home/cuckoo/.cuckoo/log/reports.log</
   location>
</localfile>
```

Codice 4.15: /var/ossec/etc/ossec.conf

cuckoo-decoder.xml in /var/ossec/etc/decoders/ sul server Wazuh aggiunge ulteriori dettagli per la decodifica del log:

```
<decoder name="cuckoo-decoder">
    <program_name>cuckoo-reporting</program_name>
    <plugin_decoder>JSON_Decoder</plugin_decoder>
</decoder>
```

Codice 4.16: cuckoo-decoder.xml

Avvio di Cuckoo e dell'interfaccia web:

```
# Inizializzazione di Cuckoo
cuckoo init

# Ricerca di aggiornamenti

cuckoo community

# Avvio dell'interfaccia web

cuckoo web --host 127.0.0.1 --port 8080 &

# Avvio di Cuckoo in modalita' debug (output verboso)

cuckoo -d
```

Codice 4.17: avvio di Cuckoo

Capitolo 5

Analisi di un data-set di malware

5.1 Invio dei campioni e struttura delle analisi

Metodi di invio: Cuckoo permette all'utente di inviare file e URL da analizzare tramite un'API, un'interfaccia web o direttamente dal server utilizzando la CLI (Command Line Interface).

```
curl -H "Authorization: Bearer S4MPL3" -F file=@/path/to/
    file http://indirizzo-server:8090/tasks/create/file
```

Codice 5.1: esempio di utilizzo dell'API Cuckoo

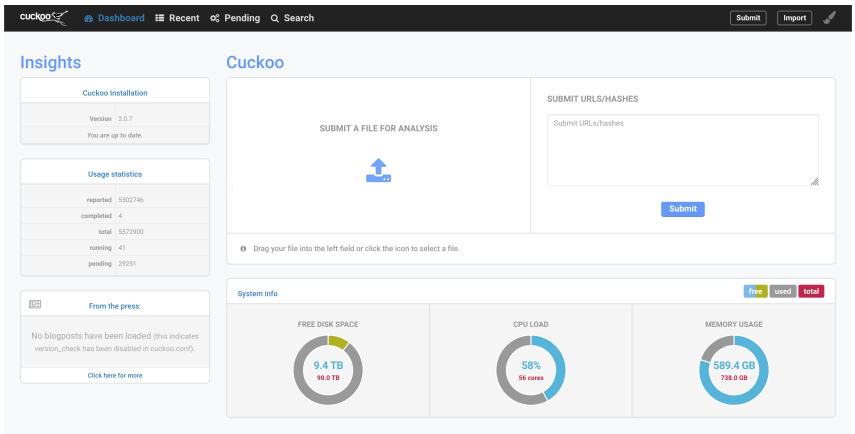

Figura 5.1: Esempio dell'interfaccia web di Cuckoo per l'invio dei campioni.

```
cuckoo submit file.exe
```

Codice 5.2: esempio di utilizzo di Cuckoo da CLI

Ricezione del campione e struttura del file report.json: Dopo l'invio, Cuckoo prende il controllo di una macchina virtuale preconfigurata (VirtualBox, KVM o VMware), ripristinandola a uno snapshot precedentemente specificato. Per un corretto funzionamento, la macchina virtuale deve avere configurato l'ultimo agente di Cuckoo, una versione di Python 2.7 e, per VirtualBox, l'interfaccia di rete deve essere impostata in modalità "host-only". È inoltre possibile personalizzare la macchina virtuale per renderla meno rilevabile dai malware, ad esempio installando software come Office o Acrobat. Il malware viene quindi eseguito all'interno della macchina virtuale, e Cuckoo raccoglie metriche dettagliate, come il traffico di rete e i dump di memoria. Questi dati sono utilizzati per l'analisi e vengono elaborati da una lista personalizzabile di moduli. Nella directory di output viene generato un archivio contenente i risultati dell'analisi su un campione reale (i file inclusi non contengono componenti dannosi per motivi di sicurezza). La directory generata da Cuckoo include i seguenti file:

- **Dump del traffico di rete**: file in formato `.pcap`.
- **Log**: relativi alle attività di analisi e ai processi di Cuckoo.
- **Dump di memoria** (se specificati).
- **Screenshot**: immagini catturate durante l'esecuzione del malware.
- **Campione analizzato** (se specificato).
- **Report dettagliato**: disponibile in formato JSON e in una versione semplificata in HTML.

Il report JSON generato da Cuckoo contiene diverse sezioni, ciascuna delle quali fornisce dettagli specifici sull'analisi effettuata:

- **info**: informazioni generali sull'analisi, come un punteggio (ancora in fase beta) che rappresenta il rischio, l'ID dell'analisi, data e ora, e i dettagli della macchina virtuale utilizzata.
- **signatures**: una lista di firme (comportamenti conosciuti e classificati come indicatori di attività malevola). Ogni firma include una famiglia di appartenenza, una breve descrizione, un punteggio di gravità e informazioni su TTP (Tactics, Techniques, and Procedures).
- **target**: informazioni aggiuntive sul campione analizzato (dimensioni, tipo, ecc.).

- **network**: un elenco completo delle comunicazioni avvenute durante l'analisi, con dettagli come indirizzi sorgente e destinazione, orari, porte utilizzate, organizzati per protocollo.

- **static**: risultati ottenuti dall'analisi statica del campione.

- **behavior**: dati sul comportamento del campione, incluse API chiamate e processi avviati.

- **debug**: informazioni di debug relative all'analisi.

- **screenshots**: screenshot catturati durante l'esecuzione del malware.

- **strings**: stringhe estratte dal campione.

- **metadata**: metadati relativi al campione analizzato.

5.2 Dettagli delle analisi svolte e informazioni estratte

Il campione utilizzato è una sezione del dataset MalwareDatabase (https://github.com/Enderma
composto da un totale di 637 malware Windows [bat, cmd, dll, doc, docm, docx, exe, iso, js, msi, vbs, xls, xlsm, xlsx]. I seguenti grafici sono stati estratti, utilizzando Python, dai risultati situati nel file report.json per ogni analisi effettuata.
In base alle necessità dell'organizzazione che decide di implementare Cuckoo è facile notare la quantità di intelligence ottenibile.

Molti di questi dati possono essere facilmente integrati con i risultati ottenuti da altri strumenti impiegati dall'università per ottenere informazioni molto più complesse. Un ottimo esempio di dati facilmente integrabili sono i dati di rete:

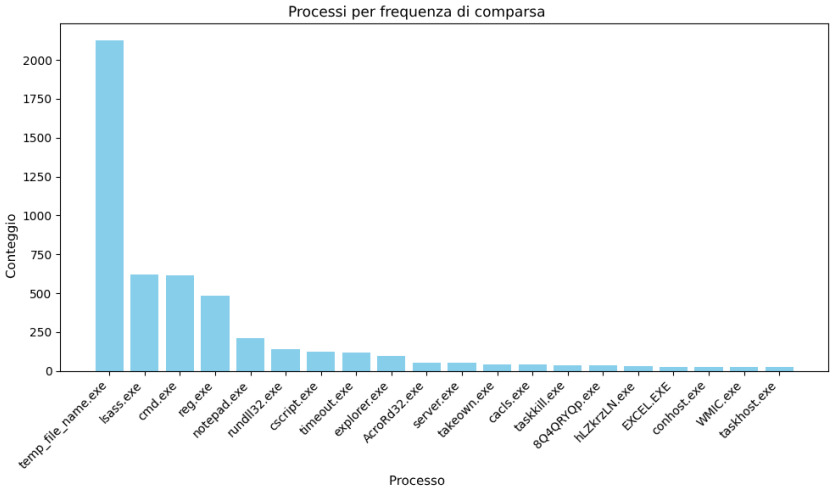

Figura 5.2: Distribuzione richieste HTTP

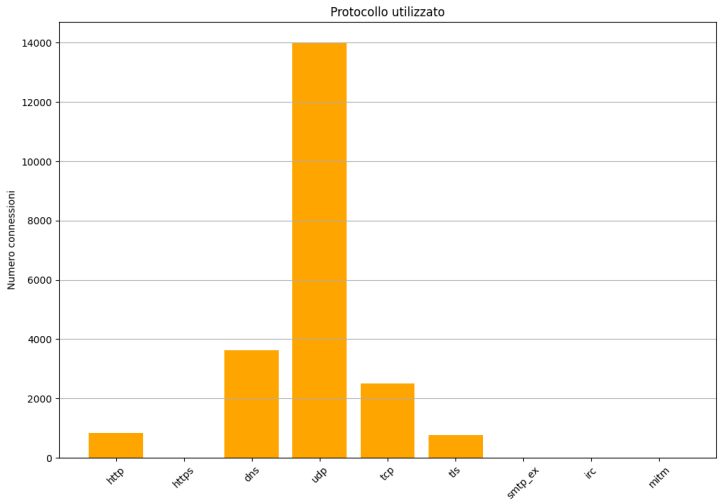

Figura 5.3: Protocolli per utilizzo

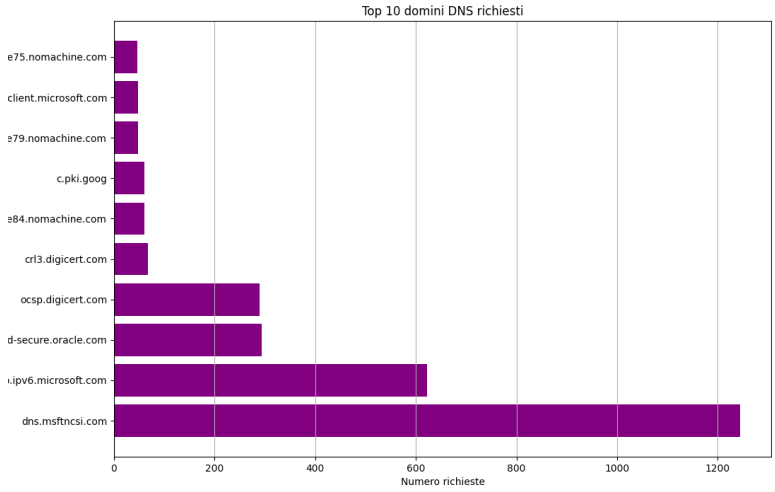

Figura 5.4: Distribuzione dei domini più contattati

In generale il corretto utilizzo dell'intelligence ottenuta consiste nel supporto decisionale per l'implementazione di misure difensive dinamiche e coerenti con le minacce moderne.

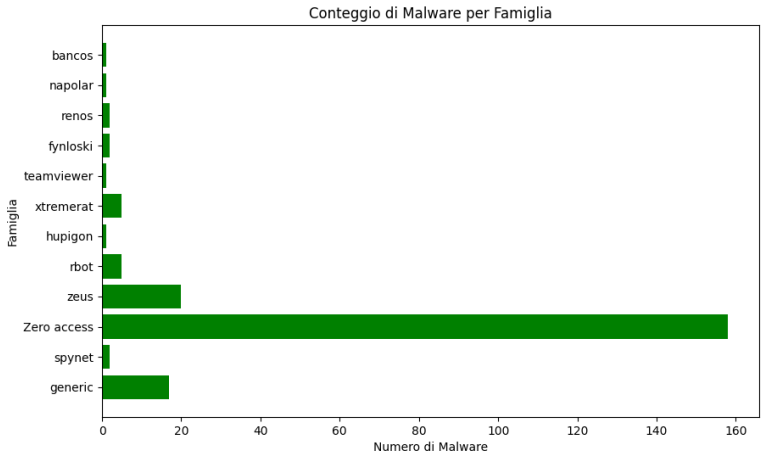

Figura 5.5: Distribuzione delle famiglie di comportamenti rilevate

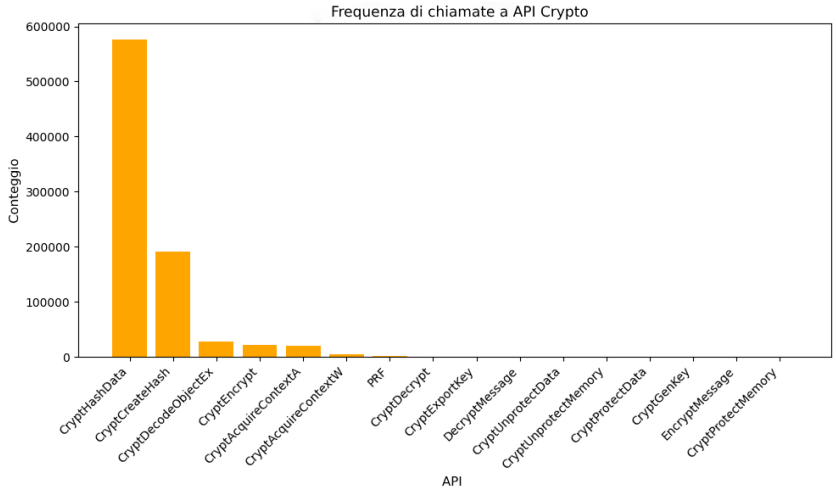

Figura 5.6: Distribuzione chiamate ad API crypto

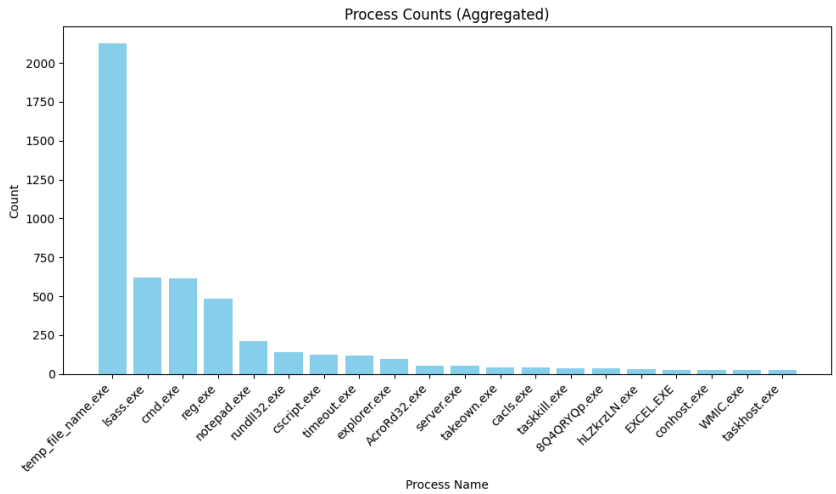

Figura 5.7: Distribuzione dei processi più frequentemente rilevati

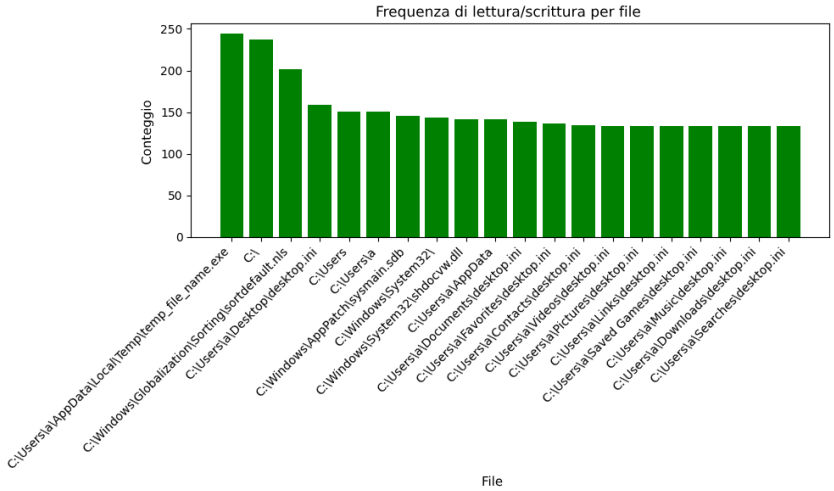

Figura 5.8: Distribuzione dei file con più letture e scritture

Capitolo 6

Risultati

Il risultato di questo progetto è la corretta inclusione di Cuckoo tra i servizi di monitoraggio della rete e delle macchine universitarie. Cuckoo, soprattutto con la prospettiva di un prossimo supporto a Python 3, si pone come uno strumento utile e altamente configurabile. Considerando le basse prestazioni e consumi dell'hardware utilizzato per questo progetto i tempi di analisi (espressi in secondi nel grafico) risultano promettenti se comparati con la mole di informazioni estratte da un singolo file.

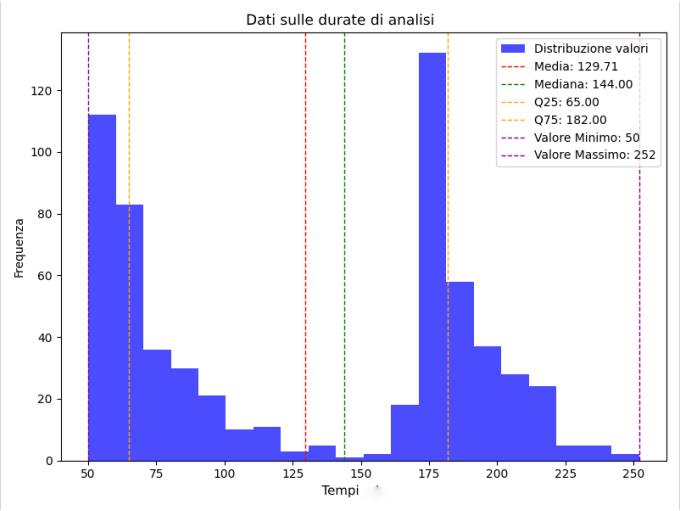

Figura 6.1: Statistiche sui tempi di analisi

La criticità attualmente più evidente risulta essere il sistema di assegnazione dei punteggi (ancora in beta). È possibile notare come capiti che dei campioni vengano valutati con punteggi superiori a 10 nonostante la scala dichiarata sia nell'intervallo [0,10].

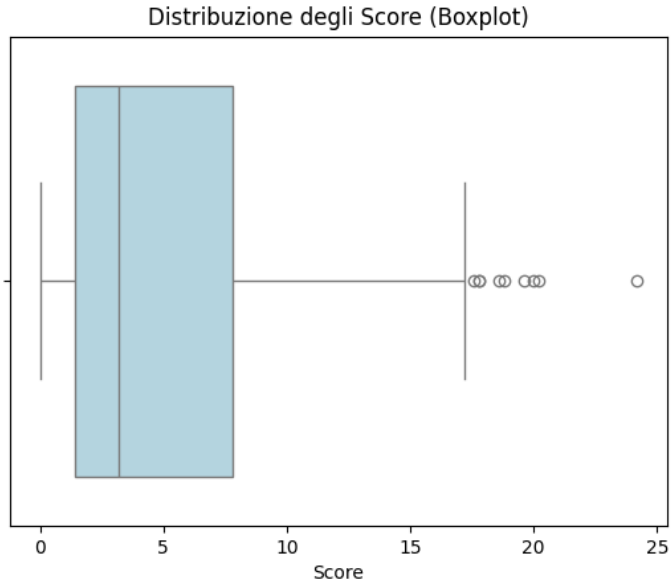

Figura 6.2: Distribuzione dei punteggi di rischio

Sebbene i falsi negativi siano pochi, la maggioranza del data set (84.1%) ha ricevuto un punteggio basso, sempre indicante il rischio di malware, ma con poche signature trovate.

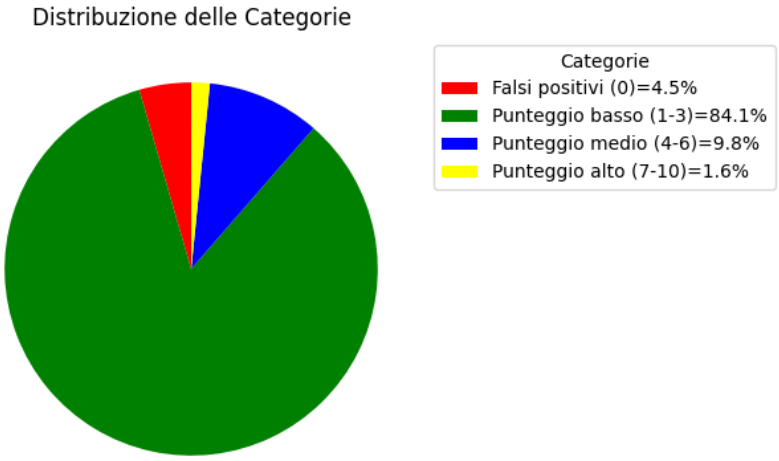

Figura 6.3: Falsi negativi e intervalli dei punteggi

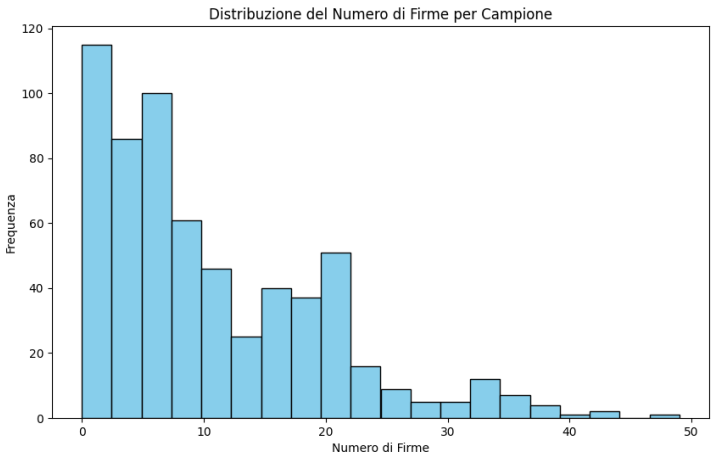

Figura 6.4: Distribuzione numero di signatures

Riguardo all'integrazione con Wazuh, i risultati mostrano un'ottima compatibilità tra i due sistemi. Gli alert funzionano correttamente aprendo la possibilità, vista la mole di informazioni, di creare più tipologie di eventi diversi (per esempio legati alla famiglia di malware oppure ai protocolli).

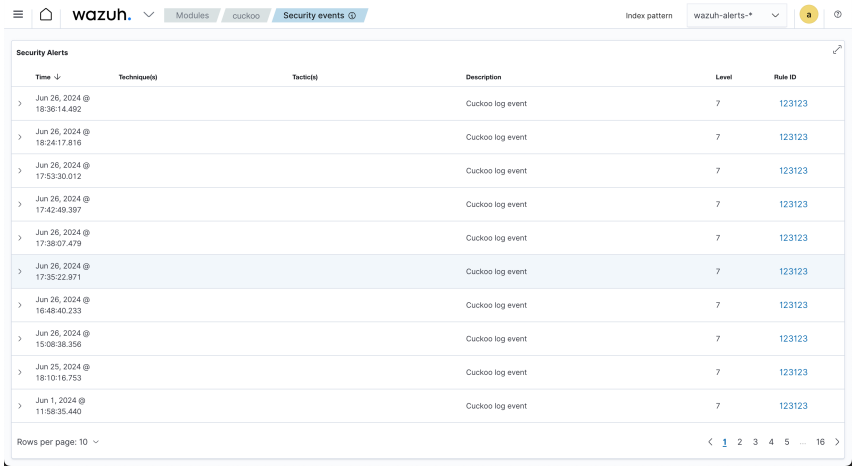

Figura 6.5: Eventi generati da Cuckoo su Wazuh

Le dashboard di Wazuh permettono la creazione di grafici anche molto complessi tramite un'interfaccia grafica di semplice utilizzo anche per utenti non direttamente esperti del settore.

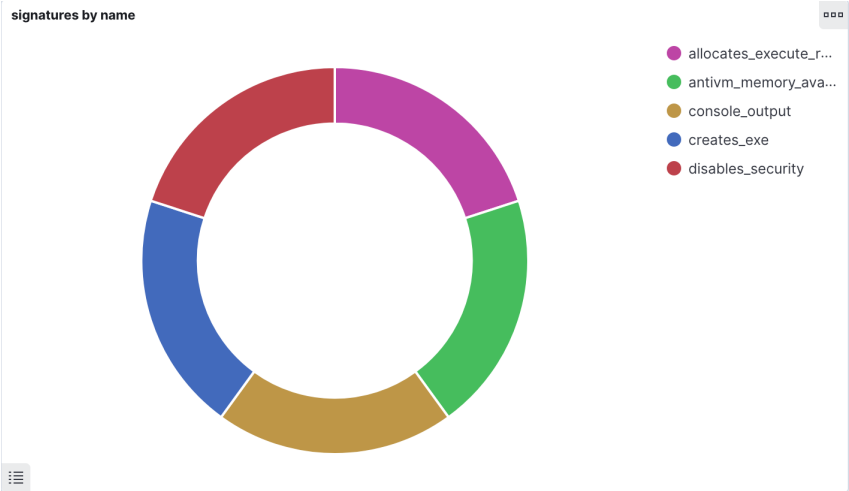

Figura 6.6: Esempio di grafico generato su Wazuh riguardante gli eventi generati da Cuckoo

La criticità trovata è legata alla dimensione delle analisi. Essendo i report spesso di dimensioni nell'ordine di GigaBytes capita frequentemente che wazuh non riesca ad assegnare (come nella sezione di alert correttamente generato qua sotto) i campi del json alle sezioni dell'alert trattandolo come un singolo blocco di testo, rendendolo inutilizzabile per ulteriori aggregazioni di dati. Questo problema emerso è probabilmente risolvibile sezionando gli eventi di Cuckoo in sotto eventi più piccoli (per esempio un evento per sezione di report.json) in modo da ridurre la dimensione media dell'evento.

Figura 6.7: Esempio di evento Cuckoo correttamente generato su Wazuh

Il monitoraggio degli eventi sull'endpoint Windows funziona come atteso e non presenta particolari criticità.

Per il futuro potrebbe essere esteso agli URL visitati (Vista la capicità di Cuckoo di analizzare anche URL).

La qualità delle informazioni estraibili ha un alto potenziale in quanto Cuckoo nella sua essenza è uno strumento facilmente scalabile e customizzabile. Come detto in precedenza l'utilizzo di questo strumento in combinazione con altre tecniche (come l'analisi statica e il monitoraggio della rete) mette a disposizione una mole di dati non trascurabile che apre le porte anche all'utilizzo di soluzioni orientate al data mining e big data. In conclusione Cuckoo Sandbox sembra essere uno strumento promettente per colmare le lacune dell'analisi statica tradizionale anche se più configurazioni andrebbero testate per una valutazione esaustiva. Anche considerare l'effettiva scalabilità dello strumento con più risorse hardware e analisi concorrenti (si possono assegnare più macchine virtuali in contemporanea) risulta doveroso per ottenere una valutazione realistica.
Oltre alle precedenti considerazioni sarebbe anche interessante esplorare la possibilità di creare pacchetti di analisi specifici per le tipologie di file più frequenti (Cuckoo permette di utilizzare pacchetti di analisi personalizzati che definiscono comportamenti specifici da mettere in atto per simulare l'utilizzo di un utente umano e che moduli utilizzare), l'utilizzo di Ansible (già utilizzato per Wazuh) per la configurazione automatizzata di un server Cuckoo e una comparazione diretta (tramite implementazione) delle altre soluzioni per l'analisi dinamica descritte in precedenza.

Bibliografia

[1] CAPEv2. Documentazione di CAPEv2. https://capev2.readthedocs.io/en/latest/installation/host/installation.html.

[2] Cuckoo Sandbox. Documentazione di Cuckoo Sandbox. https://cuckoo.readthedocs.io/en/latest/introduction/what/.

[3] PANDA. Documentazione di PANDA. https://docs.panda.re/.

[4] Wazuh. Documentazione Wazuh, 2024. https://documentation.wazuh.com/current/index.html.

[5] Wazuh. Proof of concept guide, Wazuh, 2024. https://documentation.wazuh.com/current/proof-of-concept-guide/index.html.

Ringraziamenti

Ringrazio di cuore mia madre Emanuela e mio padre Paolo per il sostegno emotivo ed economico fondamentale per la riuscita di un progetto che, dalle premesse del passato, sembrava impossibile.
Ringrazio anche la mia fidanzata Valery e i miei amici Ilaria, Giorgia e Samuele per la pazienza, la compagnia e il supporto anche nei momenti più difficili di questi anni pieni di alti e bassi. Ultimo ma non per importanza ringrazio il prof. Roberto Alfieri per la guida datami e l'aiuto cruciale negli ultimi giorni prima di questo importante giorno della mia vita.

Ancora grazie.